

Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Computer Vision and Image Processing (CSDS119)
Lab Assignment 3

Tasks:

- Read an image in Python.
- Understand that an image is a NumPy matrix.
- Identify image dimensions, channels and bit depth.
- Access and modify pixels.
- Perform basic intensity transformations.
- Visualize histograms and binary images.
- Understand the effect of resizing and noise.
- Apply basic mean and median filtering.

Part 1 — Setup

Run the following cell to import the required libraries.

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 print('Libraries imported successfully!')
```

```
Libraries imported successfully!
```

Part 2 — Upload and Read an Image

The next cell allows you to upload an image directly in Google Colab. In Jupyter, you may instead place an image in the same folder as this notebook.

```
1 # Works directly in Google Colab or local Jupyter
2 try:
3     from google.colab import files
4     uploaded = files.upload()
5     filename = next(iter(uploaded))
6 except ImportError:
7     filename = 'sample.jpg'
8
9 img = cv2.imread(filename)
10 if img is None:
11     raise FileNotFoundError('Image could not be read. Check the
12                             filename/path.')
```

```
13 plt.figure(figsize=(7, 5))
14 plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
15 plt.title('Original Image')
16 plt.axis('off')
17 plt.show()
```

Original Image



Figure 1: Original Image (sample.jpg) rendered in RGB color space.

Think Before Moving Ahead

Question: *Is Python storing this image as a photograph or as numbers?*

Answer: Python stores the image strictly as a discrete multi-dimensional numerical array (numpy.ndarray) containing numerical pixel intensities (0 to 255) across grid coordinates (y,x) and color channels. It does not store an analog photograph.

Part 3 — Explore Image Properties

```
1 print('Data type:', type(img))
2 print('Shape:', img.shape)
3 print('Data type of pixels:', img.dtype)
4 print('Minimum pixel value:', img.min())
5 print('Maximum pixel value:', img.max())
```

```
Data type: <class 'numpy.ndarray'>
Shape: (740, 740, 3)
Data type of pixels: uint8
Minimum pixel value: 0
Maximum pixel value: 255
```

Understand the Output

For a color image with shape $(height, width, 3)$:

- Height = number of rows (740)
- Width = number of columns (740)
- 3 = color channels (Blue, Green, Red in OpenCV)
- `uint8` means each channel intensity is stored using an 8-bit unsigned integer.

For an 8-bit image, $L = 2^8 = 256$, so the standard intensity range is $[0, 255]$.

Think: *Why can't a `uint8` image store the value 300?*

Answer: A `uint8` data type allocates 8 binary bits per memory cell. The maximum value that can be represented with 8 unsigned bits is $2^8 - 1 = 255$. Any numerical value exceeding 255 overflows the bit register, wrapping around modulo 256 ($300 \pmod{256} = 44$).

Part 4 — Convert the Image to Grayscale

```
1 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
2
3 plt.figure(figsize=(10, 4))
4
5 plt.subplot(1, 2, 1)
6 plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
7 plt.title('Color Image')
8 plt.axis('off')
9
10 plt.subplot(1, 2, 2)
11 plt.imshow(gray, cmap='gray')
12 plt.title('Grayscale Image')
13 plt.axis('off')
14
15 plt.tight_layout()
16 plt.show()
17
18 print('Color shape:', img.shape)
19 print('Grayscale shape:', gray.shape)
```

```
Color shape: (740, 740, 3)
Grayscale shape: (740, 740)
```



Figure 2: Side-by-side visualization of Color Image vs. Grayscale Image.

Think: *Why did the third dimension disappear after converting to grayscale?*

Answer: The third dimension in the color array corresponds to the three distinct spectral color channels (B, G, R). Converting to grayscale combines these color channels into a single scalar luminance value per pixel using the standard perceptual weighting equation:

$$Y = 0.299R + 0.587G + 0.114B \quad (1)$$

Consequently, each spatial coordinate (y, x) requires only a single scalar intensity, reducing the array shape from $(H, W, 3)$ to a 2D matrix (H, W) .

Part 5 — See the Image as a Matrix

A digital grayscale image is a matrix of intensity values.

```
1 # Grayscale image as a 2D matrix of pixel intensity values
2 print('Grayscale image slice (top-left 5x5 matrix):\n', gray[:5, :5])
```

```
Grayscale image slice (top-left 5x5 matrix):
[[254 254 254 254 254]
 [254 254 254 254 254]
 [254 254 254 254 254]
 [254 254 254 254 254]
 [254 254 254 254 254]]
```

Pixel Access

Remember: NumPy uses *[row, column]*, i.e., $[y, x]$.

```
1 # Access pixel intensity at row y=100, column x=100
2 y, x = 100, 100
3 print(f'Pixel value at (row={y}, col={x}): {gray[y, x]}')
```

```
Pixel value at (row=100, col=100): 253
```

Part 6 — Modify One Pixel

The following experiment changes only one pixel to white.

```
1 modified_img = gray.copy()
2 modified_img[100, 100] = 255
3
4 print('Original pixel value at (100, 100):', gray[100, 100])
5 print('Modified pixel value at (100, 100):', modified_img[100, 100])
6
7 plt.figure(figsize=(5, 4))
8 plt.imshow(modified_img, cmap='gray')
9 plt.title('Modified Image (One pixel changed to white)')
10 plt.axis('off')
11 plt.show()
```

```
Original pixel value at (100, 100): 253
Modified pixel value at (100, 100): 255
```

Modified Image (One pixel changed to white)



Figure 3: Modified Grayscale Image with pixel (100, 100) set to 255.

Think: *Only one number changed. Why is the change difficult to notice in the complete image?*

Answer: The image consists of $740 \times 740 = 547,600$ total pixels. A modification to a single pixel occupies less than 0.00018% of the spatial area. Furthermore, the human visual system perceives macro features, textures, and contrast gradients over neighborhoods rather than isolated single-pixel discrepancies.

Part 7 — Intensity Transformation: Negative

For an 8-bit grayscale image:

$$s = 255 - r \quad (2)$$

where r is the input intensity and s is the output intensity.

```
1 negative_img = 255 - gray
2
3 plt.figure(figsize=(10, 4))
4 plt.subplot(1, 2, 1)
5 plt.imshow(gray, cmap='gray')
6 plt.title('Grayscale Image')
7 plt.axis('off')
8
9 plt.subplot(1, 2, 2)
10 plt.imshow(negative_img, cmap='gray')
11 plt.title('Negative Image')
12 plt.axis('off')
13
14 plt.tight_layout()
15 plt.show()
```

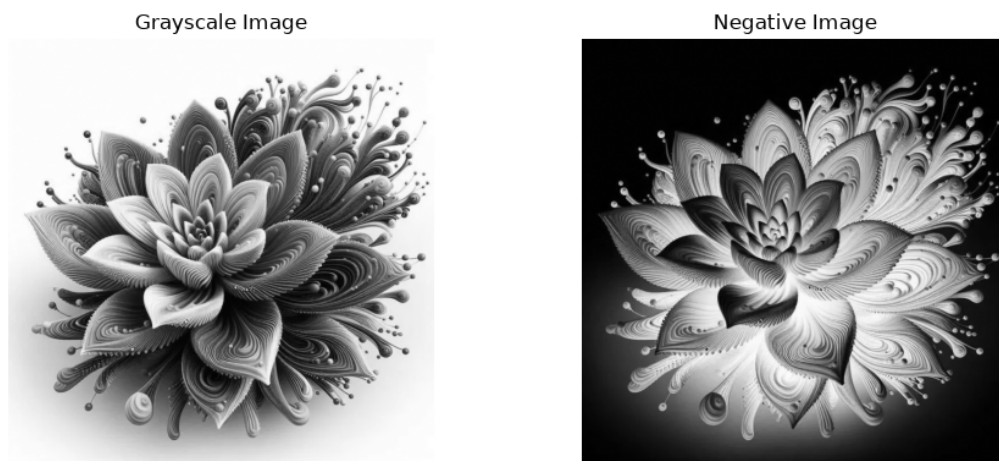


Figure 4: Original Grayscale Image compared with its Photographic Negative ($s = 255 - r$).

Think: Why does black become white and white become black?

Answer: In an 8-bit scale, pure black has intensity $r = 0$, resulting in $s = 255 - 0 = 255$ (pure white). Pure white has intensity $r = 255$, resulting in $s = 255 - 255 = 0$ (pure black). The linear transformation reflects all intensity levels across the dynamic range midpoint (127.5).

Part 8 — Brightness Transformation

Increase brightness by 50:

$$s = r + 50 \quad (3)$$

Because an 8-bit image cannot contain values above 255, clipping is required.

```
1 bright_img = np.clip(gray.astype(np.int16) + 50, 0, 255).astype(np.
    uint8)
2
3 plt.figure(figsize=(10, 4))
```

```

4 plt.subplot(1, 2, 1)
5 plt.imshow(gray, cmap='gray')
6 plt.title('Original Grayscale')
7 plt.axis('off')
8
9 plt.subplot(1, 2, 2)
10 plt.imshow(bright_img, cmap='gray')
11 plt.title('Brightened Image (+50)')
12 plt.axis('off')
13
14 plt.tight_layout()
15 plt.show()

```

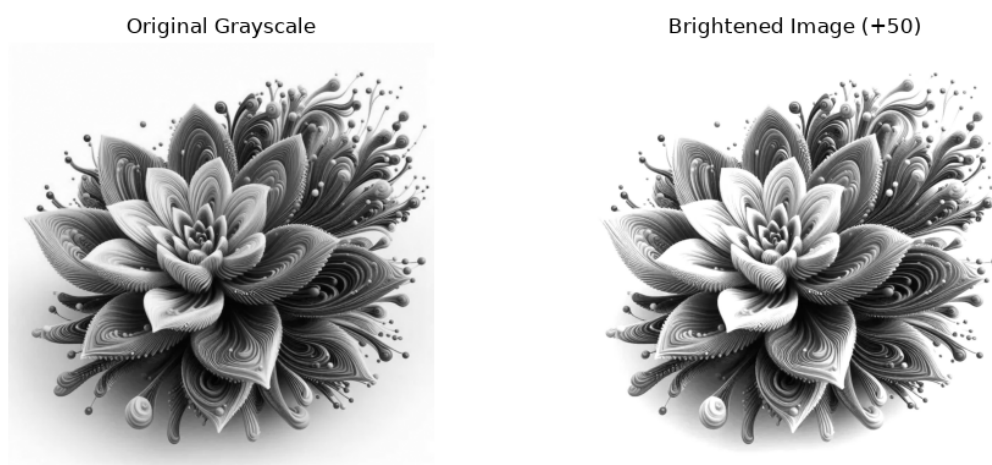


Figure 5: Original Grayscale vs. Brightened Image (+50 intensity with clipping).

Think: Why is `np.clip()` necessary here?

Answer: If an intensity value already exceeds 205, adding 50 produces a sum greater than 255. In native `uint8` arithmetic without clipping, numbers wrap around modulo 256 ($210 + 50 = 260 \equiv 4$), turning bright highlights into dark spots. Casting to a larger signed integer and applying `np.clip(..., 0, 255)` ensures values saturate at 255 rather than overflowing.

Part 9 — Thresholding

A simple threshold transforms a grayscale image into a binary image:

$$s(x, y) = \begin{cases} 255, & r(x, y) > T \\ 0, & r(x, y) \leq T \end{cases} \quad (4)$$

```

1 thresholds = [50, 100, 180, 220]
2
3 plt.figure(figsize=(14, 4))
4 for idx, T in enumerate(thresholds, 1):
5     binary_img = np.where(gray > T, 255, 0).astype(np.uint8)
6     plt.subplot(1, 4, idx)

```

```
7 plt.imshow(binary_img, cmap='gray')
8 plt.title(f'Threshold T = {T}')
9 plt.axis('off')
10
11 plt.tight_layout()
12 plt.show()
```

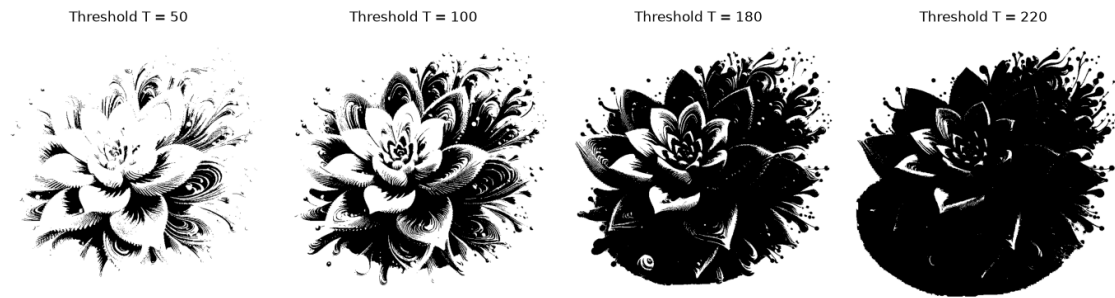


Figure 6: Binary Thresholding Results across varying thresholds $T \in \{50, 100, 180, 220\}$.

Investigation & Think: Change T to 50, 100, 180, and 220. Why does changing the threshold change the visible object/region?

Answer: The threshold parameter T determines the decision boundary for binary classification between foreground and background. At low thresholds ($T = 50$), pixels with even mild brightness become white, capturing broad regions. As T increases (180, 220), only the brightest specular highlights exceed T , causing darker and mid-tone regions to turn black.

Part 10 — Histogram

```
1 plt.figure(figsize=(7, 4))
2 plt.hist(gray.ravel(), bins=256, range=[0, 256], color='gray')
3 plt.title('Grayscale Intensity Histogram')
4 plt.xlabel('Intensity Value')
5 plt.ylabel('Pixel Count')
6 plt.grid(axis='y', alpha=0.5)
7 plt.show()
```

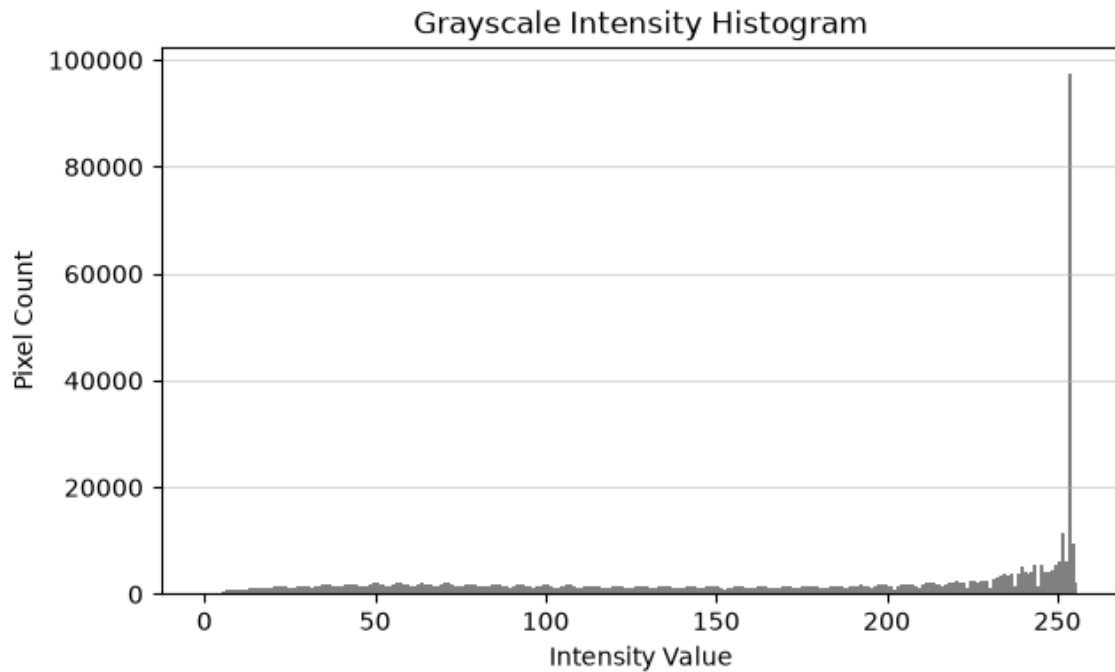


Figure 7: Grayscale Pixel Intensity Distribution Histogram.

Think: Without looking at the image, what information can the histogram tell you about the distribution of pixel intensities?

Answer: The histogram reveals the dynamic range and global illumination characteristics of the scene:

- **Brightness / Exposure:** Peaks skewed towards 0 signify an underexposed (dark) image, whereas peaks concentrated near 255 indicate overexposure.
- **Contrast:** A narrow cluster indicates low contrast, while a spread distribution across the full [0, 255] range denotes high dynamic contrast.
- **Modality:** Multiple distinct peaks imply distinct foreground/background regions suitable for segmentation.

Part 11 — Cropping and Resizing

```
1 h, w = gray.shape
2
3 # Crop central region
4 cropped_img = gray[h//4:3*h//4, w//4:3*w//4]
5
6 # Resize to half width and height
7 resized_img = cv2.resize(gray, (w // 2, h // 2))
8
9 plt.figure(figsize=(12, 4))
10 plt.subplot(1, 3, 1)
11 plt.imshow(gray, cmap='gray')
12 plt.title(f'Original Grayscale ({w}x{h})')
```

```

13 plt.axis('off')
14
15 plt.subplot(1, 3, 2)
16 plt.imshow(cropped_img, cmap='gray')
17 plt.title(f'Cropped Image ({cropped_img.shape[1]}x{cropped_img.shape
18 [0]})')
19 plt.axis('off')
20
21 plt.subplot(1, 3, 3)
22 plt.imshow(resized_img, cmap='gray')
23 plt.title(f'Resized Image ({resized_img.shape[1]}x{resized_img.shape
24 [0]})')
25 plt.axis('off')
26
27 plt.tight_layout()
28 plt.show()

```



Figure 8: Comparison of Original Image (740×740), Center Cropped Sub-matrix (370×370), and Resized Image (370×370).

Think: Which operation changes the spatial dimensions? Does resizing necessarily create new image information?

Answer: Both operations alter spatial dimensions: cropping extracts a smaller sub-matrix, reducing resolution and field of view, while resizing resamples the entire field of view onto a different grid resolution. Resizing does *not* generate new optical information; downsampling attenuates high-frequency detail, while upsampling merely interpolates existing pixel values.

Part 12 — Add Salt-and-Pepper Noise

The following creates impulse noise by randomly changing some pixels to 0 or 255.

```

1 np.random.seed(42)
2 noisy_img = gray.copy()
3 prob = 0.05 # 5% impulse noise
4
5 rnd = np.random.rand(*gray.shape)
6 noisy_img[rnd < (prob / 2)] = 0 # Pepper (0)

```

```
7 noisy_img[(rnd >= (prob / 2)) & (rnd < prob)] = 255    # Salt (255)
8
9 plt.figure(figsize=(6, 5))
10 plt.imshow(noisy_img, cmap='gray')
11 plt.title('Salt-and-Pepper Noisy Image')
12 plt.axis('off')
13 plt.show()
```



Figure 9: Image corrupted with 5% Salt-and-Pepper Impulse Noise.

Part 13 — Mean vs Median Filtering

Now connect the experiment to order-statistic filtering.

```
1 # Mean filter using 3x3 window
2 mean_filtered = cv2.blur(noisy_img, (3, 3))
3
4 # Median filter using 3x3 window
5 median_filtered = cv2.medianBlur(noisy_img, 3)
6
7 plt.figure(figsize=(14, 5))
8 plt.subplot(1, 3, 1)
9 plt.imshow(noisy_img, cmap='gray')
10 plt.title('Noisy Image')
11 plt.axis('off')
12
13 plt.subplot(1, 3, 2)
14 plt.imshow(mean_filtered, cmap='gray')
15 plt.title('Mean Filtered (3x3)')
16 plt.axis('off')
17
```

```
18 plt.subplot(1, 3, 3)
19 plt.imshow(median_filtered, cmap='gray')
20 plt.title('Median Filtered (3x3)')
21 plt.axis('off')
22
23 plt.tight_layout()
24 plt.show()
```

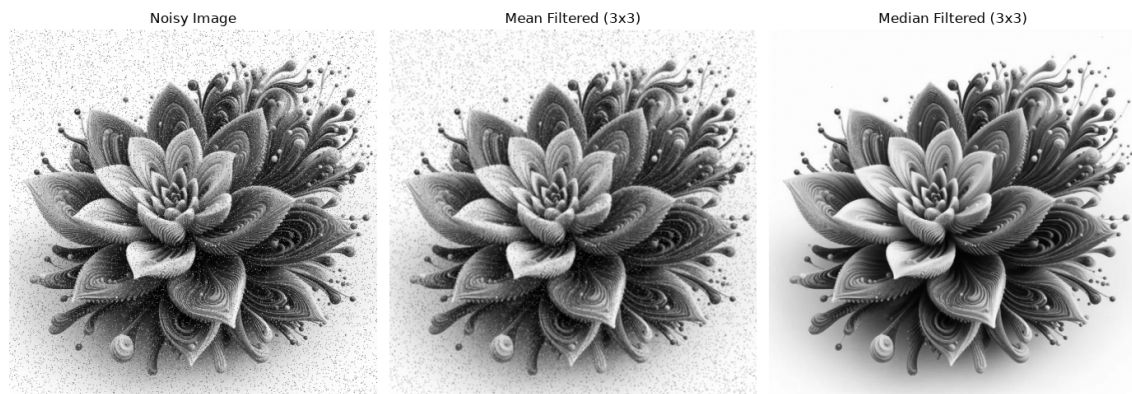


Figure 10: Comparison of Impulse Noisy Image, Linear Mean Filtered Image (3×3), and Order-Statistic Median Filtered Image (3×3).

Critical Thinking

Compare the noisy image, mean-filtered image and median-filtered image:

1. **Which filter removes the salt-and-pepper noise more effectively?**

Answer: The **median filter** removes salt-and-pepper noise far more effectively. Because impulse noise consists of extreme outliers (0 and 255), sorting the neighborhood ranks these outliers at the extremes, leaving a representative median value that restores the true underlying intensity.

2. **Which filter preserves edges better?**

Answer: The **median filter** preserves sharp edges significantly better. Linear mean filtering averages across edge boundaries, creating blur and halo artifacts, whereas the median filter maintains step discontinuities.

3. **Why can an extreme value such as 255 strongly affect the mean but have much less effect on the median?**

Answer: The mean is a linear sum $\frac{1}{N} \sum x_i$. An extreme outlier (255) pulls the arithmetic mean heavily towards the outlier. In contrast, the median is an order-statistic rank metric that depends solely on the middle value of the sorted sequence, completely ignoring the magnitude of tail outliers.

4. **What do you expect if the window is changed from 3×3 to 7×7 ?**

Answer: A 7×7 window increases the neighborhood size from 9 to 49 pixels. For the mean filter, this causes severe image blurring and loss of high-frequency detail. For the median filter, a 7×7 window can suppress higher noise densities but causes rounding of fine corners and erosion of thin lines.

Part 14 — Manual Order-Statistic Exercise

Consider the following 3×3 neighborhood:

$$\mathbf{N} = \begin{bmatrix} 42 & 43 & 44 \\ 43 & 255 & 45 \\ 44 & 45 & 46 \end{bmatrix} \quad (5)$$

Step 1: Sorted Neighborhood Sequence

Arranging the 9 intensity values in ascending order:

$$\{42, 43, 43, 44, \mathbf{44}, 45, 45, 46, 255\} \quad (6)$$

Step 2: Manual Mathematical Calculations

- **Minimum:** $\min = 42$
- **Maximum:** $\max = 255$
- **Median:** The 5th element in the 9-element sorted list is 44.
- **Mean:**

$$\mu = \frac{42 + 43 + 44 + 43 + 255 + 45 + 44 + 45 + 46}{9} = \frac{607}{9} \approx 67.44 \quad (7)$$

- **Midpoint:**

$$\text{Midpoint} = \frac{\min + \max}{2} = \frac{42 + 255}{2} = \frac{297}{2} = 148.5 \quad (8)$$

Statistic Metric	Theoretical Formula	Calculated Value
Minimum	$\min(\mathbf{N})$	42
Maximum	$\max(\mathbf{N})$	255
Median	$\text{median}(\mathbf{N})$	44
Mean	$\frac{1}{9} \sum_{i=1}^9 x_i$	67.44
Midpoint	$\frac{\min + \max}{2}$	148.5

Table 1: Summary of Order-Statistic Values for the 3×3 Neighborhood Matrix.

Step 3: Python Execution Verification

```

1 neighborhood = np.array([
2     [42, 43, 44],
3     [43, 255, 45],
4     [44, 45, 46]
5 ])
6
7 print('Neighborhood Matrix:\n', neighborhood)
8 print('---')
9 print('Minimum: ', np.min(neighborhood))
10 print('Maximum: ', np.max(neighborhood))
11 print('Median: ', np.median(neighborhood))
12 print('Mean: ', round(float(np.mean(neighborhood)), 2))
13 print('Midpoint:', (int(np.max(neighborhood)) + int(np.min(neighborhood))) / 2.0)

```

Neighborhood Matrix:

```
[[ 42  43  44]
 [ 43 255  45]
 [ 44  45  46]]
```

```
Minimum: 42
Maximum: 255
Median: 44.0
Mean: 67.44
Midpoint: 148.5
```